

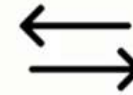
Les opérateurs d'agrégation MongoDB



Ça veut dire quoi !

PyMongo

Perform different
operations



(CRUD)



Create , Read , Update , Delete

PyMongo



Python application



MongoDB

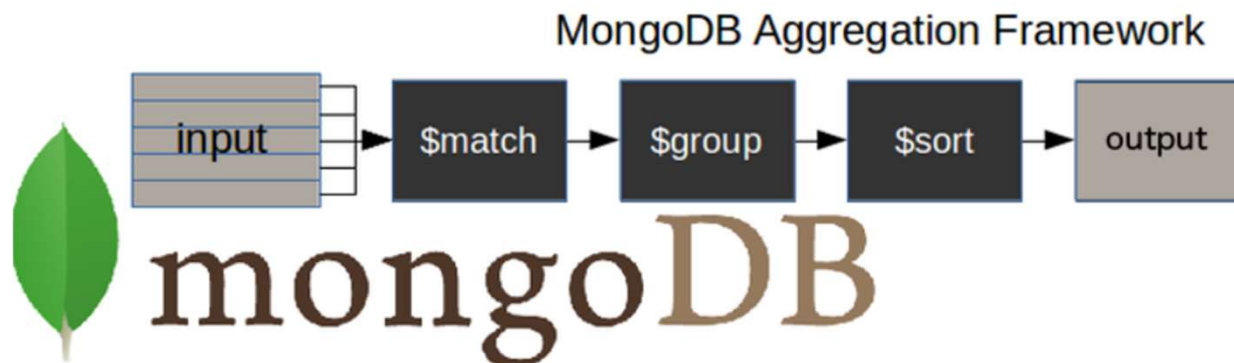


Qu'est-ce que l'agrégation dans MongoDB ?

L'**agrégation** est une manière de traiter un grand nombre de documents dans une collection en les faisant passer par différentes étapes. Ces étapes constituent ce que l'on appelle un *pipeline*. Les étapes d'un pipeline peuvent *filtrer*, *trier*, *grouper*, *remodeler* et *modifier* les documents qui passent par le pipeline.

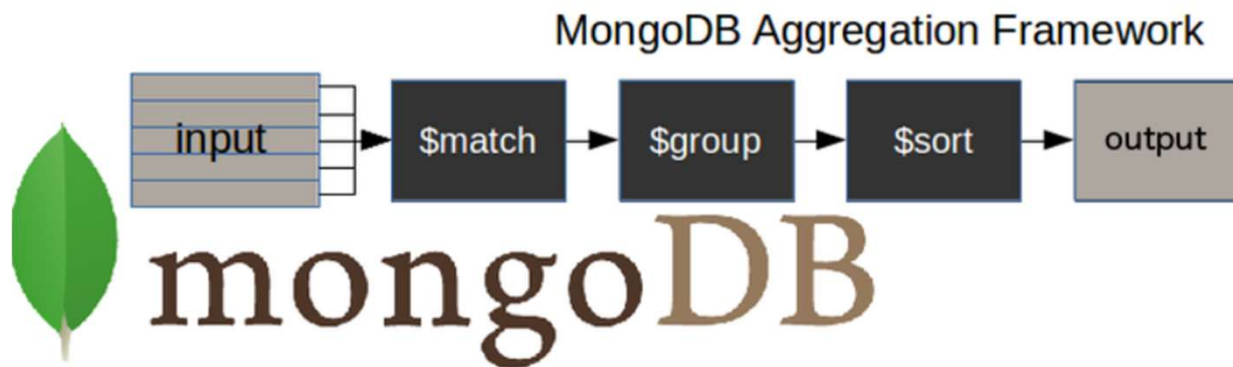
Comment fonctionne le pipeline d'agrégation MongoDB ?

Voici un diagramme illustrant un pipeline d'agrégation MongoDB typique.



Comment fonctionne le pipeline d'agrégation MongoDB ?

Voici un diagramme illustrant un pipeline d'agrégation MongoDB typique.



- ✓ **\$match** : filtre les documents avec lesquels nous devons travailler, ceux qui répondent à nos besoins
- ✓ **\$group** : effectue le travail d'agrégation
- ✓ **\$sort** : trie les documents résultants de la manière que nous souhaitons (par ordre croissant ou décroissant).

Syntaxe: `db.collectionName.aggregate(pipeline, options)`

Où *option* est un paramètre optionnel

Les opérateurs d'agrégation MongoDB: Exemple

Une requête d'agrégation est de la forme:

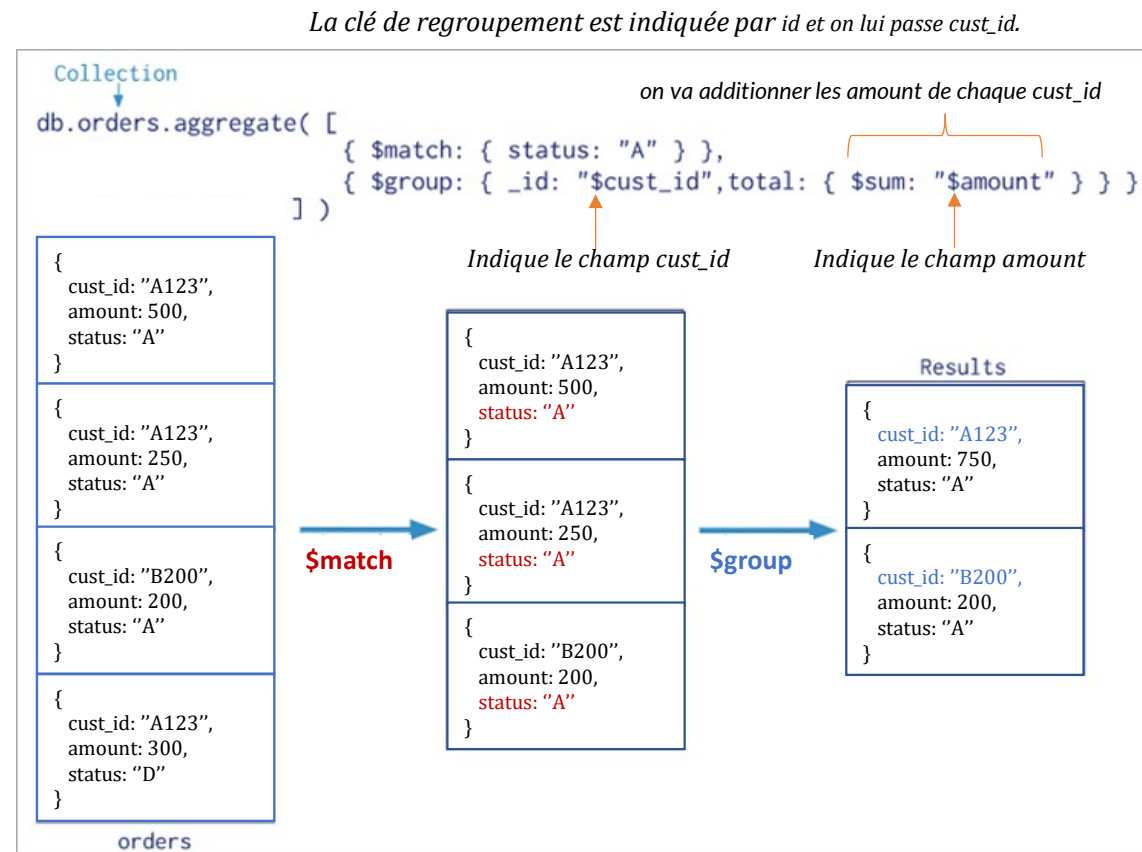
```
db.col_name.aggregate( [  
  {...},  
  {...},  
  ....  
])
```

L'opération d'agrégation sélectionne les documents dont le *status*="A", ensuite regroupe les documents résultants par le champ *cust_id* (clé de regroupement *_id*) et calcule le total des *amount* par *cust_id* en utilisant la méthode *sum*.

Orders

```
{  
  cust_id: ...,  
  amount: ...,  
  status: ...  
}
```

L'opérateur **\$match** sert pour filtrer les documents d'une collection



Calcul du montant total par client et pour le status A.

Exemples de documents

University.json

```
{
  "country": "Spain",
  "city": "Salamanca",
  "name": "USAL",
  "location": {
    "type": "Point",
    "coordinates": [ -5.6722512, 17, 40.9607792 ]
  },
  "students": [
    { "year": 2014, "number": 24774 },
    { "year": 2015, "number": 23166 },
    { "year": 2016, "number": 21913 },
    { "year": 2017, "number": 21715 }
  ]
},
{
  "country": "Spain",
  "city": "Salamanca",
  "name": "UPSA",
  "location": {
    "type": "Point",
    "coordinates": [ -5.6691191, 17, 40.9631732 ]
  },
  "students": [
    { "year": 2014, "number": 4788 },
    { "year": 2015, "number": 4821 },
    { "year": 2016, "number": 6550 },
    { "year": 2017, "number": 6125 }
  ]
}
```

resto.json

```
{
  "_id": 1, "quartier": "q1", "cuisine": "Couscous",
  "grades": [
    { "note": "B", "score": 21, "annee": 2023 },
    { "note": "A", "score": 7, "annee": 2022 },
    { "note": "B", "score": 27, "annee": 2020 }
  ],
  "nom": "Resto 5 étoiles", "restaurant_id": 100
},
{
  "_id": 2, "quartier": "q2", "cuisine": "Riz au poisson",
  "grades": [
    { "note": "B", "score": 11, "annee": 2023 },
    { "note": "C", "score": 46, "annee": 2021 }
  ],
  "nom": "Amitié", "restaurant_id": 200
},
{
  "_id": 3, "quartier": "q1", "cuisine": "Macaroni",
  "grades": [
    { "note": "B", "score": 11, "annee": 2023 },
    { "note": "B", "score": 17, "annee": 2022 }
  ],
  "nom": "Bonne cuisine", "restaurant_id": 300
}
```

Les opérateurs d'agrégation MongoDB

- ✓ Nbre total d'universités:

```
db.University.aggregate([{$count:"total"}])
```



Resultat:

total: 2

Ou bien

```
db.University.aggregate([  
  { $group : { _id : null, total_docs : { $sum : 1 } } } ])
```

- *_id: null: Indique à MongoDB de regrouper tous les documents en un seul groupe.*
- *total_docs: { \$sum: 1 } : Pour chaque document du groupe (ici, tous les documents), on ajoute 1 à la variable total_docs. Cela compte donc le nombre total de documents.*

- ✓ Le nombre de documents par université:

```
db.University.aggregate([  
  { $group : { _id : "$name", total_docs : { $sum : 1 } } }  
])
```



Resultat:

```
{ "_id" : "UPSA", "totaldocs" : 1 }  
{ "_id" : "USAL", "totaldocs" : 1 }
```

Equivalent de Count() de SQL*

- ✓ Nombre total d'entrées dans tous les tableaux students:

```
db.University.aggregate([  
  { $unwind : "$students" },  
  { $count : "total_documents" }  
])
```



Resultat:

```
{ "total_documents" : 8 }
```

\$unwind transforme un tableau en plusieurs documents en dupliquant le document initial pour chaque élément du tableau.

- ✓ Le nombre d'étudiants sur les 4 ans par université

```
db.University.aggregate([
  { $unwind: "$students" },
  { $group: { "_id": { "nom": "$name" }, "nbStudents": { $sum: "$students.number" } } }
])
```

Resultat:

```
{ "_id" : nom:"UPSA", " nbEtudiants " : 22284 }
{ "_id" : nom:"USAL", " nbEtudiants " : 91568 }
```

Select sum(number) as nbStudents
From University
Group by name;

- ✓ Les universités qui ont plus de 30 000 étudiants sur les 4 ans : ➔ *il faut rajouter un filtre*

```
db.University.aggregate([
  { $unwind: "$students" },
  { $group: { "_id": { "nom": "$name" }, "nbStudents": { $sum: "$students.number" } } },
  ➔ { $match: { "nbStudents": { $gt: 30000 } } }
])
```

← Having en SQL

Select sum(number) as nbStudents
From University
Group by name
Having nbStudents>30000;

- ✓ Groupement par *name* et *city*:



```
{ $group: { "_id": { "nom": "$name", "city": "$city" }, "totalStudents": { $sum: "$students.number" } } }
```

- ✓ Pour récupérer les résultats d'une requête

```
db.University.aggregate([  
  { $unwind: "$students" },  
  { $group: { "_id": { "nom": "$name" }, "totalStudents": { $sum: "$students.number" } } },  
  { $out: "res" }  
])
```

Ensuite on peut faire **db.res.find()**

```
db.University.aggregate([  
  { $unwind: "$students" },  
  { $group: { "_id": null, nb: { $sum: "$students.number" } } }  
])
```

Tous les docs sont regroupés ensemble:
 $22284 + 91568 = 113852$


{ **_id**: null, **nb**: 113852 }

Préparation pour les exercices

Créer une base de données **agregation** et importer les fichiers ci-dessous dans des collections qui ont les mêmes noms que les fichiers:

- ✓ university.json
- ✓ resto.json
- ✓ notes.json
- ✓ courses.json
- ✓ users.json
- ✓ materiels.json

Exemple:

```
mongoimport --db agregation --collection university --file university.json --jsonArray
```

- ✓ Ecrire une requête qui affiche seulement le *name*, *city*, *country* et *students* des universités



```
[
  {
    country: 'Spain',
    city: 'Salamanca',
    name: 'USAL',
    students: [
      { year: 2014, number: 24774 },
      { year: 2015, number: 23166 },
      { year: 2016, number: 21913 },
      { year: 2017, number: 21715 }
    ]
  },
  {
    country: 'Spain',
    city: 'Salamanca',
    name: 'UPSA',
    students: [
      { year: 2014, number: 4788 },
      { year: 2015, number: 4821 },
      { year: 2016, number: 6550 },
      { year: 2017, number: 6125 }
    ]
  }
]
```

Les fonctions min, max, avg et sort

notes.json

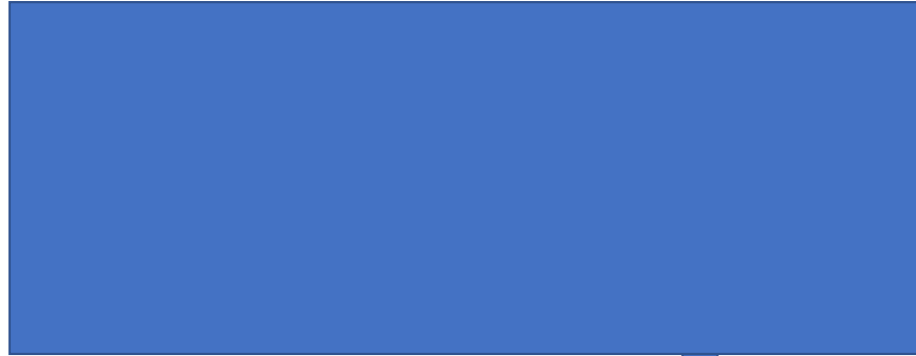
```
[
  { "_id": 1, "idEtudiant": 1, "note": 10, "idMatiere": 1 },
  { "_id": 2, "idEtudiant": 2, "note": 12, "idMatiere": 2 },
  { "_id": 5, "idEtudiant": 4, "note": 9, "idMatiere": 1 },
  { "_id": 6, "idEtudiant": 1, "note": 8, "idMatiere": 3 },
  { "_id": 8, "idEtudiant": 3, "note": 15, "idMatiere": 1 },
  { "_id": 9, "idEtudiant": 1, "note": 16, "idMatiere": 2 },
  { "_id": 7, "idEtudiant": 2, "note": 14, "idMatiere": 3 },
  { "_id": 10, "idEtudiant": 2, "note": 12, "idMatiere": 1 },
  { "_id": 4, "idEtudiant": 4, "note": 11.5, "idMatiere": 2 },
  { "_id": 3, "idEtudiant": 3, "note": 10.5, "idMatiere": 3 }
]
```

```
db.notes.aggregate([
  { $group : { _id : null,
    note_min : { $min : "$note" },
    note_max : { $max : "$note" },
    note_moy : { $avg : "$note" }
  }
}]
```



```
{ _id: null,
  note_min: 8,
  note_max: 16,
  note_moy: 11.8
}
```

- ✓ Afficher les moyennes des trois meilleurs étudiants



}}



```
SELECT avg(note) as note_moy
FROM notes
GROUP BY idEtudiant
ORDER BY note_moy desc
LIMIT 3
```

```
[
  { _id: { matricule: 3 }, note_moy: 12.75 },
  { _id: { matricule: 2 }, note_moy: 12.666666666666666 },
  { _id: { matricule: 1 }, note_moy: 11.333333333333334 }
]
```

Jointure entre documents

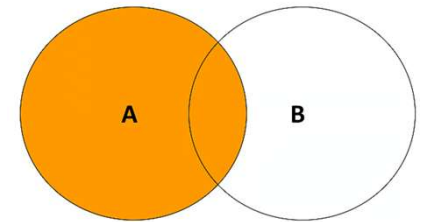
University.json

```
{
  "country": "Spain",
  "city": "Salamanca",
  "name": "USAL",
  "location": {
    "type": "Point",
    "coordinates": [ -5.6722512, 17, 40.9607792 ]
  },
  "students": [
    { "year": 2014, "number": 24774 },
    { "year": 2015, "number": 23166 },
    { "year": 2016, "number": 21913 },
    { "year": 2017, "number": 21715 }
  ]
},
{
  "country": "Spain",
  "city": "Salamanca",
  "name": "UPSA",
  "location": {
    "type": "Point",
    "coordinates": [ -5.6691191, 17, 40.9631732 ]
  },
  "students": [
    { "year": 2014, "number": 4788 },
    { "year": 2015, "number": 4821 },
    { "year": 2016, "number": 6550 },
    { "year": 2017, "number": 6125 }
  ]
}
```

Courses.json

```
{
  "university": "USAL",
  "name": "Computer Science",
  "level": "Excellent"
},
{
  "university": "USAL",
  "name": "Electronics",
  "level": "Intermediate"
},
{
  "university": "UPSA",
  "name": "Communication",
  "level": "Excellent"
}
```

Jointure entre documents



On va utiliser **\$lookup**

est l'équivalent d'un **LEFT JOIN** en SQL

```
db.C1.aggregate([ {  
  $lookup:  
  {  
    from: C2,  
    localField: <the join field from the input collection C1>,  
    foreignField: <the join field from the collection C2 >,  
    as: <alias>  
  }  
}]
```

```
SELECT *, <alias>  
FROM C1 LEFT JOIN C2 ON  
        C2.foreignField = C1.localField;
```

Jointure entre documents

Question: Afficher pour chaque université, ses info et ses cours.

On utilise *\$lookup*

Reponse:

```
db.University.aggregate([
  {
    $lookup:
      {
        from: "Courses",
        localField: "name",
        foreignField: "university",
        as: "res"
      }
  }
])
```

```
SELECT *
FROM University LEFT JOIN Course
ON University. name= Course. university
```

output

Jointure entre documents

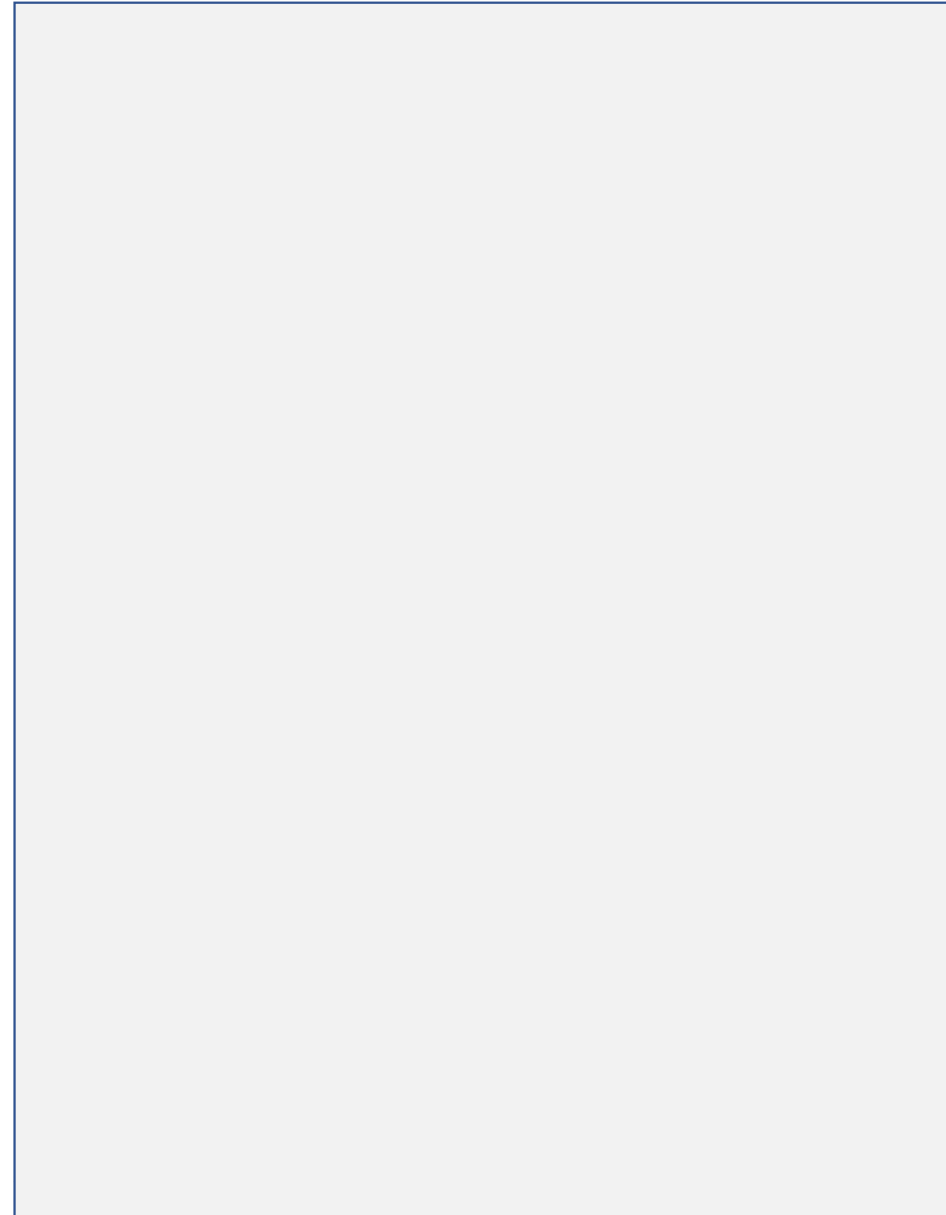
output

```
{
  _id: ObjectId("63f4ec8207901e1c903484d3"),
  country: 'Spain',
  city: 'Salamanca',
  name: 'USAL',
  location: { type: 'Point', coordinates: [ -5.6722512, 17, 40.9607792 ] },
  students: [
    { year: 2014, number: 24774 },
    { year: 2015, number: 23166 },
    { year: 2016, number: 21913 },
    { year: 2017, number: 21715 }
  ],
  res: [
    {
      _id: ObjectId("63f61db02be0a8baf84535ef"),
      university: 'USAL',
      name: 'Electronics',
      level: 'Intermediate'
    },
    {
      _id: ObjectId("63f61db02be0a8baf84535f1"),
      university: 'USAL',
      name: 'Computer Science',
      level: 'Excellent'
    }
  ]
},
```

```
{
  _id: ObjectId("63f4ec8207901e1c903484d4"),
  country: 'Spain',
  city: 'Salamanca',
  name: 'UPSA',
  location: { type: 'Point', coordinates: [ -5.6691191, 17, 40.9631732 ] },
  students: [
    { year: 2014, number: 4788 },
    { year: 2015, number: 4821 },
    { year: 2016, number: 6550 },
    { year: 2017, number: 6125 }
  ],
  res: [
    {
      _id: ObjectId("63f61db02be0a8baf84535f0"),
      university: 'UPSA',
      name: 'Communication',
      level: 'Excellent'
    }
  ]
}
```

Quel est le résultat de cette requete ?

```
db.University.aggregate([
  { $lookup: {
    from: "Courses",
    localField: "name",
    foreignField: "university",
    as: "res"
  } },
  { $project: {country:1, city:1, name:1, _id:0,
    students:1, "res.name":1, "res.level":1 } }
])
```



User.json

```
[
  {"_id":1, "name":"Ahmed", "country":"RIM"},
  {"_id":2, "name":"Fati", "country":"Senegal"},
  {"_id":3, "name":"Oumar", "country":"Maroc"},
  {"_id":4, "name":"Aicha", "country":"RIM"},
  {"_id":5, "name":"Ali", "country":"Algerie"}
]
```

materiel.json

```
[
  {"_id":1, "user_id":1,"produit":"Laptop","prix":1000},
  {"_id":2, "user_id":2,"produit":"Phone","prix":500},
  {"_id":3, "user_id":3,"produit":"Tablet","prix":700},
  {"_id":4, "user_id":1,"produit":"Datashow","prix":800},
  {"_id":5, "user_id":4,"produit":"Mouse","prix":20}
]
```

1. Pour chaque utilisateur, afficher la liste de matériels achetés

users			
_id	int	*	
name	str	*	
country	str	*	

materiels			
_id	int	*	
user_id	int	*	
produit	str	*	
prix	int	*	

SELECT *

FROM user LEFT JOIN materiels

ON user. id= materiels.user_id;

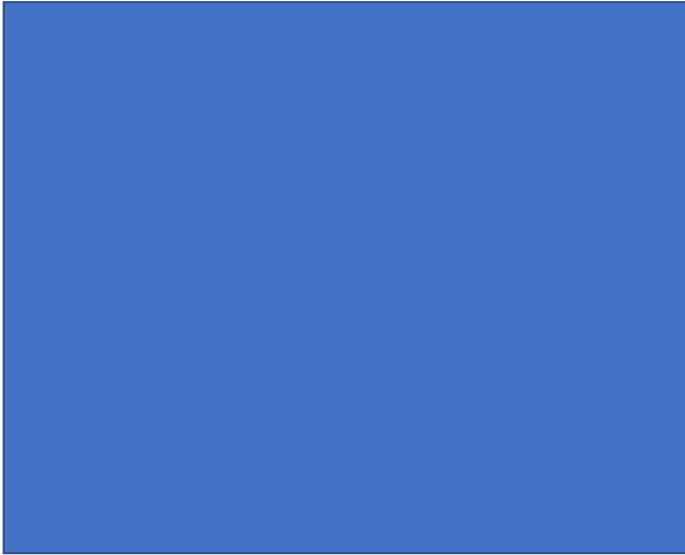
See the expected output



Résultat

```
[
  {
    _id: 1,
    name: 'Ahmed',
    country: 'RIM',
    output: [
      { _id: 1, user_id: 1, produit: 'Laptop', prix: 1000 },
      { _id: 4, user_id: 1, produit: 'Datashow', prix: 800 }
    ]
  },
  {
    _id: 2,
    name: 'Fati',
    country: 'Senegal',
    output: [ { _id: 2, user_id: 2, produit: 'Phone', prix: 500 } ]
  },
  {
    _id: 3,
    name: 'Oumar',
    country: 'Maroc',
    output: [ { _id: 3, user_id: 3, produit: 'Tablet', prix: 700 } ]
  },
  {
    _id: 4,
    name: 'Aicha',
    country: 'RIM',
    output: [ { _id: 5, user_id: 4, produit: 'Mouse', prix: 20 } ]
  },
  {
    _id: 5,
    name: 'Ali',
    country: 'Algerie',
    output: [] }
]
```

2. La même question mais en gardant seulement le nom de l'utilisateur et la liste de son matériel



```
[
  {
    name: 'Ahmed',
    List_produits: [
      { _id: 1, user_id: 1, produit: 'Laptop', prix: 1000 },
      { _id: 4, user_id: 1, produit: 'Datashow', prix: 800 }
    ]
  },
  {
    name: 'Fati',
    List_produits: [ { _id: 2, user_id: 2, produit: 'Phone', prix: 500 } ]
  },
  {
    name: 'Oumar',
    List_produits: [ { _id: 3, user_id: 3, produit: 'Tablet', prix: 700 } ]
  },
  {
    name: 'Aicha',
    List_produits: [ { _id: 5, user_id: 4, produit: 'Mouse', prix: 20 } ]
  },
  { name: 'Ali', List_produits: [] }
]
```

User.json

```
[
  {"_id":1, "name":"Ahmed", "country":"RIM"},
  {"_id":2, "name":"Fati", "country":"Senegal"},
  {"_id":3, "name":"Oumar", "country":"Maroc"},
  {"_id":4, "name":"Aicha", "country":"RIM"},
  {"_id":5, "name":"Ali", "country":"Algerie"}
]
```

materiel.json

```
[
  {"_id":1, "user_id":1,"produit":"Laptop","prix":1000},
  {"_id":2, "user_id":2,"produit":"Phone","prix":500},
  {"_id":3, "user_id":3,"produit":"Tablet","prix":700},
  {"_id":4, "user_id":1,"produit":"Datashow","prix":800},
  {"_id":5, "user_id":4,"produit":"Mouse","prix":20}
]
```

3. Afficher la liste de matériels achetés par des utilisateurs RIMIens



```
[
  {
    _id: 1,
    name: 'Ahmed',
    country: 'RIM',
    output: [
      { _id: 1, user_id: 1, produit: 'Laptop', prix: 1000 },
      { _id: 4, user_id: 1, produit: 'Datashow', prix: 800 }
    ]
  },
  {
    _id: 4,
    name: 'Aicha',
    country: 'RIM',
    output: [ { _id: 5, user_id: 4, produit: 'Mouse', prix: 20 } ]
  }
]
```

Exemples de documents

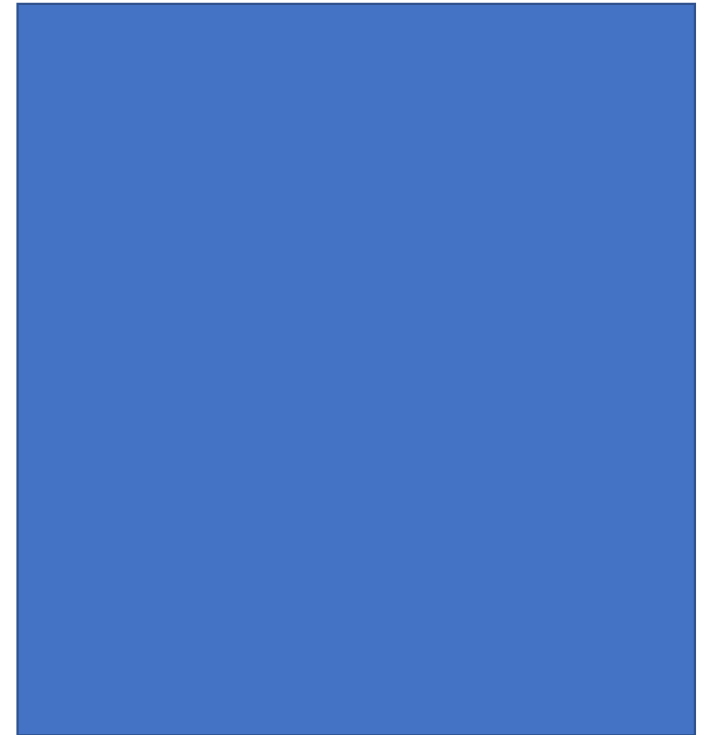
resto.json

```
{
  "_id": 1, "quartier": "q1", "cuisine": "Couscous",
  "grades" : [
    {"note": "B", "score": 21, "annee": 2023},
    {"note": "A", "score": 7, "annee": 2022},
    {"note": "B", "score": 27, "annee": 2020}
  ],
  "nom": "Resto 5 étoiles", "restaurant_id": 100
},
{
  "_id": 2, "quartier": "q2", "cuisine": "Riz au poisson",
  "grades" : [
    {"note": "B", "score": 11, "annee": 2023},
    {"note": "C", "score": 46, "annee": 2021}
  ],
  "nom": "Amitié", "restaurant_id": 200
},
{
  "_id": 3, "quartier": "q1", "cuisine": "Macaroni",
  "grades" : [
    {"note": "B", "score": 11, "annee": 2023},
    {"note": "B", "score": 17, "annee": 2022}
  ],
  "nom": "Bonne cuisine", "restaurant_id": 300
}
```

Exercice: fichier resto.json

Restaurants de "q1" avec leurs notes de 2023

```
[  
  { cuisine: 'Couscous', nom: 'Resto 5 étoiles', score: 21, note: 'B' },  
  { cuisine: 'Macaroni', nom: 'Bonne cuisine', score: 11, note: 'B' }  
]
```



Classement des cuisines par score moyen

```
[  
  { nbRestaurants: 2, cuisine: 'Riz au poisson', moyenneScore: 28.5 },  
  { nbRestaurants: 3, cuisine: 'Couscous', moyenneScore: 18.33 },  
  { nbRestaurants: 2, cuisine: 'Lacciri', moyenneScore: 16.5 },  
  { nbRestaurants: 4, cuisine: 'Macaroni', moyenneScore: 12.5 }  
]
```



Meilleure note par restaurant

```
[  
  { meilleureNote: 12, quartier: 'q4', nom: 'Mc Donald' },  
  { meilleureNote: 17, quartier: 'q1', nom: 'Bonne cuisine' },  
  { meilleureNote: 18, quartier: 'q3', nom: 'Beldo' },  
  { meilleureNote: 27, quartier: 'q1', nom: 'Resto 5 étoiles' },  
  { meilleureNote: 46, quartier: 'q2', nom: 'Amitié' }  
]
```

